

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ  
ГИДРОМЕТЕОРОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»  
(РГГМУ)**

---

**Факультет гидрометеорологического обеспечения экономико-  
управленческой деятельности в отраслях и комплексах**

**Кафедра**

**Направление подготовки  
09.03.03  
«Прикладная информатика»**

**«Отчет по заданию (учебная практика)»**

**Выполнил: Рахимкулов Д. Р.,  
курс – 2,  
группа – ПИ-Б24-1**

**Руководитель: Сафонова Т.В.  
к.т.н., доцент кафедры  
Прикладной  
информатики**

**Санкт-Петербург, 2026 г.**

## СОДЕРЖАНИЕ

1. Введение
2. Описание постановки задачи
3. Выбор среды разработки и технологий
4. Проектирование структуры приложения
5. Реализация класса Product
6. Реализация класса Inventory
7. Реализация графического интерфейса пользователя
8. Логика работы программы
9. Тестирование программы
10. Заключение
11. Список использованных источников

## ВВЕДЕНИЕ

В современных информационных системах задачи учета товаров, материалов и складских запасов являются одними из наиболее распространенных. Даже для небольших организаций важно иметь инструмент, позволяющий быстро добавлять товары, изменять их параметры, удалять ненужные позиции и рассчитывать общую стоимость текущего запаса.

Целью данной работы являлась разработка приложения “Продуктовый инвентарь”, предназначенного для ведения учета продуктов. Программа реализована на языке программирования C++ в среде разработки CLion на операционной системе macOS с использованием фреймворка Qt6.11.0.

Основной задачей являлось создание удобного приложения с графическим интерфейсом, в котором пользователь может:

- Добавлять новые продукты;
- Удалять существующие позиции;
- Изменять данные о продукте;
- Просматривать список товаров;
- Рассчитывать суммарную стоимость всего инвентаря.

В процессе разработки особое внимание было уделено удобству интерфейса, корректности обработки пользовательского ввода и логическому разделению программы на классы.

## ОПИСАНИЕ ПОСТАНОВКИ ЗАДАЧИ

Согласно условию задания требовалось:

- Создать класс “Product”, содержащий:
  1. Цену;
  2. Номер;
  3. Количество.
- Создать класс “Inventory”, который:
  1. Хранит список продуктов;
  2. Выполняет учет различных продуктов;
  3. Рассчитывает общую стоимость инвентаря.

Также было принято решение реализовать графический интерфейс пользователя, чтобы работа с программой была удобнее по сравнению с консольным вариантом.

## ВЫБОР СРЕДЫ РАЗРАБОТКИ И ТЕХНОЛОГИЙ

### Почему был выбран язык C++?

В первую очередь, C++ подходит для выполнения задания по следующим критериям:

- Поддержка объектно-ориентированного программирования;
- Высокая производительность;
- Удобная работа с классами и структурами данных;

- Широкое применение в разработке приложений.

Так как задание напрямую связано с созданием классов, использование C++ выглядит наиболее логичным решением.

### **Почему был выбран фреймворк Qt?**

Фреймворк Qt был выбран, потому что он предоставляет:

- Удобные средства создания графического интерфейса;
- Готовые компоненты: кнопки, таблицы, поля ввода, окна сообщений;
- Встроенный механизм сигналов и слотов;
- Хорошая совместимость с macOS.

### **Почему был выбран CLion?**

Среда разработки (IDE) CLion удобна благодаря:

- Качественной подсветке синтаксиса;
- Встроенной работе с CMake;
- Удобной навигации по проекту;
- Отладчику;
- Хорошей поддержке Qt-проектов.

## **ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПРИЛОЖЕНИЯ**

Для упрощения разработки программа была разделена на несколько логических частей:

- “Product” – описание одного товара;
- “Inventory” – управление списком товаров;
- “MainWindow” – графический интерфейс;
- “main.cpp” – точка входа в приложение.

Такое разделение соответствует принципам объектно-ориентированного программирования, когда каждый класс отвечает только за собственную область задач.

## **РЕАЛИЗАЦИЯ КЛАССА “PRODUCT”**

Класс “Product” предназначен для хранения данных об одном товаре.

### **Поля класса**

В классе используются три закрытых поля:

- “m\_id” – уникальный номер продукта;
- “m\_price” – цена;
- “m\_quantity” – количество.

Закрытый доступ был выбран для защиты данных от некорректного изменения извне.

### **Конструкторы**

```
// “Product::Product(): m_id(0), m_price(0.0), m_quantity(0) {}”
```

Конструктор по умолчанию создает пустой объект.

```
// "Product::Product(int id, double price, int quantity)"
```

Параметризованный конструктор позволяет сразу создать готовый объект товара.

### **Методы доступа**

Для работы с полями реализованы геттеры и сеттеры:

- getId(), setId();
- getPrice(), setPrice();
- getQuantity(), setQuantity();

Это обеспечивает инкапсуляцию данных.

## **РЕАЛИЗАЦИЯ КЛАССА "INVENTORY"**

Класс "Inventory" отвечает за хранение всех товаров.

### **Почему был выбран Qlist?**

Для хранения продуктов использован контейнер "Qlist<Product>", потому что:

- Он хорошо интегрирован с Qt;
- Удобен для хранения объектов;
- Поддерживает добавление и удаление элементов;
- Прост в использовании.

### **Добавление товара**

```
// "void Inventory::addProduct(const Product &product)"
```

Перед добавлением выполняется проверка, существует ли уже товар с таким ID. Это необходимо для сохранения уникальности номера продукта.

### **Удаление товара**

```
// "bool Inventory::removeProduct(int id)"
```

Метод ищет товар по идентификатору и удаляет его.

Возвращение true/false показывает, успешно ли выполнена операция.

### **Изменение товара**

```
// "bool Inventory::updateProduct(int id, const Product &newProduct)"
```

Позволяет заменить данные существующего товара.

### **Получение списка товаров**

```
// "Qlist<Product> Inventory::getProducts() const"
```

Возвращает текущий список продуктов.

### **Расчет общей стоимости**

```
// "double Inventory::totalValue() const"
```

Суммируется стоимость каждого товара: (Цена \* количество).

После этого возвращается общий итог.

## РЕАЛИЗАЦИЯ ГРАФИЧЕСКОГО ИНТЕРФЕЙСА ПОЛЬЗОВАТЕЛЯ

Интерфейс реализован в классе MainWindow.

### Почему использован “QMainWindow”?

Класс “QMainWindow” удобен как основа приложения, поскольку предоставляет:

- Главное окно программы;
- Центральную область размещения элементов;
- Поддержку меню и статусной строки при необходимости.

### Таблица товаров

Для отображения списка использован “QTableWidget”.

Причины выбора:

- Простое отображение строк и столбцов;
- Возможность выделения строк;
- Быстрое обновление содержимого.

Таблица содержит три столбца:

- ID;
- Цена;
- Количество.

### Поля ввода

Использованы три объекта “QLineEdit”:

- Ввод ID;
- Ввод цены;
- Ввод количества.

Для удобства добавлены подсказки (placeholderText)

### Кнопки управления

Использованы кнопки:

- Добавить;
- Удалить;
- Изменить.

### Метка общей стоимости

Использован “QLabel”, отображающий итоговую стоимость всего инвентаря.

## ЛОГИКА РАБОТЫ ПРОГРАММЫ

### Добавление товара

При нажатии кнопки выполняются действия:

1. Считываются данные из полей;
2. Проверяется корректность значений;
3. Проверяется уникальность ID;
4. Создается объект “Product”;
5. Объект добавляется в “Inventory”;
6. Таблица обновляется.

### Почему реализована валидация?

Без проверки пользователь мог бы ввести:

- Отрицательную цену;
- Отрицательное количество;
- Текст вместо числа;
- Повторяющийся ID;

Поэтому применена защита через “QMessageBox”.

### **Удаление товара**

Пользователь выбирает строку таблицы и нажимает кнопку удаления. По ID запись удаляется из списка.

### **Изменение товара**

При выборе строки данные автоматически подставляются в поля ввода.

После редактирования пользователь нажимает кнопку изменения.

Это удобно, поскольку не нужно вводить значения заново.

### **Автоматическое обновление таблицы**

После каждой операции вызывается:

```
// “refreshTable();”
```

Это позволяет синхронизировать интерфейс и внутренние данные.

## **ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ**

В процессе проверки были протестированы следующие сценарии:

### **Корректная работа**

- Добавление нескольких товаров;
- Расчет общей стоимости;
- Изменение цены и количества;
- Удаление товара.

### **Проверка ошибок**

- Ввод текста вместо числа;
- Ввод отрицательной цены;
- Ввод отрицательного количества;
- Попытка добавить товар с существующим ID.

Во всех случаях программа корректно выводила предупреждение.

## **ЗАКЛЮЧЕНИЕ**

В ходе выполнения работы было разработано приложение “Продуктовый инвентарь” на языке программирования C++ с использованием Qt6.11.0.

В программе реализованы все требования задания:

- Создан класс “Product”;
- Создан класс “Inventory”;
- Организован учет продуктов;
- Реализован расчет общей стоимости;

- Создан удобный графический интерфейс.

Дополнительно были реализованы:

- Проверка корректности ввода;
- Автоматическое обновление таблицы;
- Защита от дублирования ID.

Использование объектно-ориентированного подхода позволило сделать код структурированным, понятным и удобным для дальнейшего развития. В будущем приложение можно дополнить сохранением данных в файл, поиском товаров и сортировкой списка.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Официальная документация Qt.
2. Справочные материалы по Qt Widgets.
3. Книга – C++20 Пол Дейтел, Харви Дейтел.
4. Документация CLion JetBrains.

Исходный код программы можно найти по ссылке - [https://github.com/call-CloudY9/Product\\_Inventory](https://github.com/call-CloudY9/Product_Inventory).